Document No: P1401r1
Date: 2019-06-16
Author: Andrzej Krzemieński
Reply-to: akrzemi1 (at) gmail (dot) com
Audience: EWG

Narrowing contextual conversions to `bool`

This paper proposes to allow conversions from integral types to type `bool` in *contextually converted constant expression of type* `bool`.

Tony tables

Today	If accepted
`if constexpr(bool(flags & Flags::Exec))`	`if constexpr(flags & Flags::Exec)`
`if constexpr(flags & Flags::Exec != 0)`	`if constexpr(flags & Flags::Exec)`
`static_assert(bool(N));`	`static_assert(N);`
`static_assert(N % 4 != 0);`	`static_assert(N % 4);`

Revisions

R0 -> R1

Extended the discussion and problem analysis. Outlined the reange of possible changes. Not proposing wording anymore: the goal is to obtain the direction from EWG first.

Motivation

Clang currently fails to compile the following program, and this behavior is Standard-compliant:

```
enum Flags { Write = 1, Read = 2, Exec = 4 };

template <Flags flags>
int f() {
  if constexpr (flags & Flags::Exec) // fails to compile due to narrowing
    return 0;
  else
    return 1;
```

```
}

int main() {
  return f<Flags::Exec>(); // when instantiated like this
}
```

This is because, as currently specified, narrowing is not allowed in contextual conversion to `bool` in core constant expressions. If compilers were standard-compliant, even the following code would be ill-formed.

```
template <std::size_t N>
class Array
{
  static_assert(N, "no 0-size Arrays");
  // ...
};

Array<16> a; // fails to compile in pure C++
```

All these situations can be fixed by typing a `static_cast` to type `bool` or comparing the result to 0, but the fact remains that this behavior is surprising. For instance, using run-time equivalents of the above constructs compiles and executes fine:

```
if (flags & Flags::Exec) // ok
  {}
```

```
assert(N); // ok
```

Note that the proposal only affects the contextual conversions to `bool`: it does not affect implicit conversions to `bool` in other contexts.

Background

The no-narrowing requirement was added in [CWG 2039], which indicates it was intentional. However, the issue documentation does not state the reason.

The no-narrowing requirement looks justified in the context of `noexcept` specifications, where the "double `noexcept`" syntax is so strange that it can be easily misused. For instance, if I want to say that my function `f()` has the same `noexcept` specification as function `g()`, it doesn't seem impossible that I could mistakenly type this as:

```
void f() noexcept(g);
```

To the uninitiated this looks reasonable; and it compiles. Also, if `g()` is a `constexpr` function, the following works as well:

```
void f() noexcept(g());
```

The no-narrowing requirement helps minimize these bugs, so it has merit. But other contexts, like `static_assert`, are only victims thereof.

Analysis

Affected contexts

The definition of *contextually converted constant expression of type* `bool` ([expr.const]/7) is used in four places in the standard:

- `static_assert`
- `if constexpr`
- `explicit(bool)`
- `noexcept(bool)`

Note that `requires`-clause does not use the definition, as it requires that the expression "shall be a constant expression of type `bool`" ([temp.constr.atomic]/3). The problems caused by the *contextually converted constant expression of type* `bool` are mostly visible in the first two cases. In case of `explicit(bool)` we expext a type trait to be used as an expression.

Similarly, in the case of `noexcept(bool)` we only expect a type trait or a `noexcept`-expression.

The following table illustrates where compilers allow a conversion to bool in a *contextually converted constant expression of type* `bool` against the C++ requirements:

context	gcc	clang	icc	msvc
`static_assert`	yes	yes	yes	yes
`if constexpr`	yes	no	yes	yes*
`explicit(bool)`	no	no	–**	–**
`noexcept(bool)`	no	yes	yes	yes*

* MSVC accepts the code but issues a warning even in `/W1`.
** Feature not implemented.

Accepting this proposal would be to some extent standardizing the existing practice among compiler vendors.

Types contextually convertible to type `bool`

The following table lists types that are contextually convertible to type `bool`:

type	**allowed in constant expr**	**`true` when**
class with conversion to `bool`	yes	operator returns `true`
class with conversion to a built-in type convertibel to `bool`	as per rules below	as per below rules
object/function pointer	no	not null
function name/reference	no	always
array name/reference	no	always
pointer to member	no	not null
integral type	no, except for 0 and 1	not zero
floating-point type	no	not plus/minus zero
nullptr_t	no	never
unscoped enumeration	no, except for 0 and 1	not zero

The problem, which this proposal is trying to fix, has only been reported when conversions from integral types or unscoped enumeraiotn types are involved, as for these types such conversion has practical and often used meaning:

- non-empty collection/sequence,
- is a given bit in a bitmask set.

We have never encountered a need to check if a floating-point value is exactly +/-0 in this way. Technically, checking a pointer has a meaning: “is it really pointing to some object/function”, but it is more difficult to imagine a practical use case for it in *contextually converted constant expression of type* `bool`.

Implicit conversions to `bool`

Some have suggested that a conversion to `bool` in general should not be considered narrowing, that `bool` should not be treated as a small integral type, and that the conversion to `bool` should be

treated as a request to classify the states of the object as one of the two categories.

We do not want to go there. Even this seemingly correct reasoning can lead to bugs like this one:

```
struct Container {
  explicit Container(bool zeroSize, bool zeroCapacity) {}
  explicit Container(size_t size, int* begin) {}
};

int main() {
  std::vector<int> v;
  X x (v.data(), v.size()); // bad order!
}
```

If the feature that prevents narrowing conversions can detect this bug, we would like to use this opportunity.

Implicit conversions to `bool` in constant expressions

Another situation brought up while discussing this problem was if the following code should be correct:

```
// a C++03 type trait
template <typename T>
struct is_small {
  enum { value = (sizeof(T) == 1) };
};

template <bool small> struct S {};
S<is_small<char>::value> s; // int 1 converted to bool
```

In constant expressions the situation is different, because whether a conversion is narrowing or not depends not only on the types but also on the velaues, which are known at compile-time. We think that after [P0907r4] was adopted, and `bool` has a well defined representation, conversion from `1` to `true` is now defined as non-narrowing.

Compatibility with C `assert()` macro

As described in [LWG 3011], currently macro `assert()` from the C Standard Library only allows expressions of scalar types, and does not support the concept of expressions “contextually converted to `bool`”. We believe that this issue does not interact with our proposal. For instance, the following example works even under the C definition of macro `assert()`:

```
template <std::size_t N>
auto makeArray()
{
  assert(N);
  // ...
}
```

But it stops working if we change `assert(N)` to `static_assert(N)`.

How to fix this

There is a two-dimensional space of possible solutions to this problems with two extremal solutions being:

1. Leave the specification as it is: no narrowing is allowed. (This leaves all the known compilers non-conformant.)
2. Just allow any implicit conversions in *contextually converted constant expression of type* `bool`. (This compromizes the solution in [CWG 2039], whatever its intent was.)

The two “degrees of freedom” in the solution space are:

1. Apply the fix only in the subset of the four contexts where the definition of *contextually converted constant expression of type* `bool` is used; e.g., only in `static_assert` and `if constexpr`.
2. Allow conversion to `bool` only from a subset of types implicitly convertible to `bool`, e.g., only integral and scoped enumeration types.

In fact, relaxing only `static_assert` and `if constexpr`, only for integral and scoped enumeration types solves all issues that have been reported by users that we are aware of.

We request guidance from EWG on which approach to adopt.

Acknowledgements

Jason Merrill originally reported this issue in CWG reflector. Tomasz Kamiński reviewed the paper and suggested improvements.

Members of EWGI significantly improved the quality of the paper.

References

[CWG 2039] Richard Smith, “Constant conversions to `bool`”.

[LWG 3011] Jonathan Wakely, “Requirements for `assert(E)` inconsistent with C”.

[WG14 N2207] Martin Sebor, Jonathan Wakely, Florian Weimer, “Assert Expression Problematic For C++”.

[P0907r4] JF Bastien, “Signed Integers are Two’s Complement”.